

MINISTRY OF EDUCATION AND RESEARCH OF THE REPUBLIC OF MOLDOVA

Technical University of Moldova

Faculty of Computers, Informatics and Microelectronics

Department of Software Engineering and Automation

REPORT

Laboratory Work No. 6

Operating Systems

Inside the xv6 File System

Student: Bianca-Andreea Tataroi
Group: FAF-242

Checked by: Reițman Patricia

Chișinău – 2026

Contents

1 Objective **2**

2 Laboratory Environment **2**

3 Part 1 – Exploring the xv6 File System **3**
3.1 Listing Files and Directories 3
3.2 Creating an Empty Directory 3
3.3 Part 1 – Observations 5

4 Part 2 – Implementing mystat and Hard Links **6**
4.1 Creating the mystat Program 6
4.2 Adding mystat to the Makefile 7
4.3 Testing mystat 7
4.4 Creating a Hard Link 8
4.5 Removing the Original Filename 9
4.6 Examining Link Count Changes 9
4.7 Examining the On-Disk Inode Structure 10
4.8 Part 2 – Observations 10

5 Part 3 – Measuring File System Limits **11**
5.1 Examining File System Constants 11
5.2 Calculating Maximum File Size 11
5.3 Examining the Disk Layout 12
5.4 Testing Long Filenames 13
5.5 Part 3 – Observations 14

6 Part 4 – Implementing Recursive Find **14**
6.1 Creating the find Program 14
6.2 Explanation of the Program 16
6.3 Adding find to the Makefile 16
6.4 Creating the Test Directory Tree 17
6.5 Testing Recursive File Search 17
6.6 Testing Directory Search 18
6.7 Part 4 – Observations 18

7 Results and Discussion **19**

8 GitHub Repository **19**

9 Conclusion **20**

1 Objective

The objective of this laboratory work was to explore the internal organization of the xv6 file system and understand how an operating system manages files, directories, inodes, hard links, and disk blocks.

The laboratory was completed using the xv6 operating system running through QEMU inside an Ubuntu Linux virtual machine.

The following topics were investigated:

- xv6 file and directory types;
- inode numbers and directory entries;
- file metadata using the `stat()` call;
- hard links and link counts;
- the on-disk inode structure;
- block sizes and file system limitations;
- maximum filename and file sizes;
- disk layout and metadata allocation;
- recursive directory traversal.

Two user programs were implemented in C: `mystat.c` and `find.c`.

2 Laboratory Environment

The laboratory was performed using Ubuntu Linux inside VirtualBox.

The xv6 source code was located in:

```
cd ~/xv6-riscv
pwd
```

The working directory was:

```
/home/bianca/xv6-riscv
```

The xv6 operating system was started using:

```
make qemu
```

This command compiled the necessary components and launched the RISC-V operating system using QEMU.

After startup, the following message appeared:

```
xv6 kernel is booting

hart 1 starting
hart 2 starting
init: starting sh
$
```

The dollar sign represented the xv6 shell prompt.

The source files were edited using the Nano text editor, and the programs were compiled using the RISC-V cross-compilation toolchain.

3 Part 1 – Exploring the xv6 File System

3.1 Listing Files and Directories

The first task was to inspect the contents of the xv6 root directory.

The following command was executed:

```
ls
```

The output included:

```
. 1 1 1024
.. 1 1 1024
README 2 2 2441
cat 2 3 36728
echo 2 4 35592
console 3 25 0
```

Each line contains four fields:

1. File or directory name;
2. File type;
3. Inode number;
4. Size in bytes.

The file type values are defined in `kernel/stat.h`.

Type	Meaning
1	Directory
2	Regular file
3	Device

The root directory had inode number 1.

The entries `.` and `..` both referenced inode 1 because the root directory is its own parent.

3.2 Creating an Empty Directory

A new directory named `a` was created:

```
mkdir a
ls a
```

The output was:

```
. 1 26 32
.. 1 1 1024
```

The new directory received inode number 26.
Its size was 32 bytes because it contained two directory entries:

- `.` – the current directory;
- `..` – the parent directory.

Each directory entry occupies 16 bytes.
The structure is defined in `kernel/fs.h`:

```
#define DIRSIZ 14

struct dirent {
    ushort inum;
    char name[DIRSIZ];
};
```

The inode number occupies 2 bytes, while the name field occupies 14 bytes.
Therefore:

```
2 + 14 = 16 bytes

2 entries * 16 bytes = 32 bytes
```

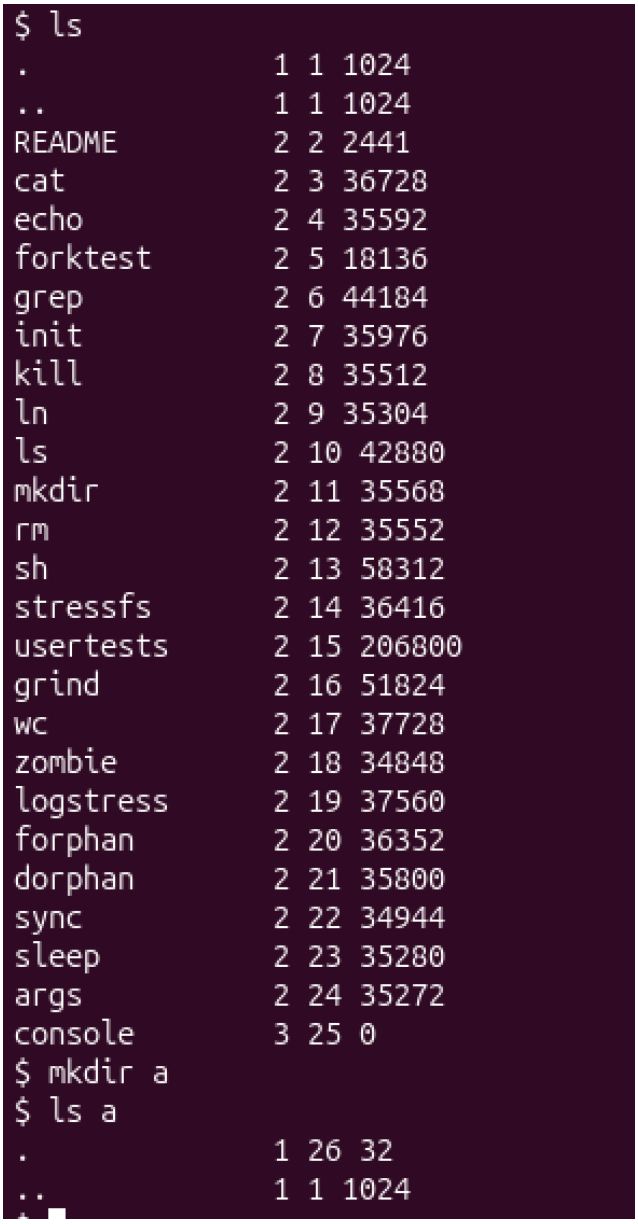


Figure 1: Exploring xv6 directories and creating an empty directory

3.3 Part 1 – Observations

1. xv6 distinguishes directories, regular files, and devices using numeric type values.
2. The root directory has inode number 1.
3. The entries `.` and `..` both reference inode 1 in the root directory.
4. An empty directory has a size of 32 bytes because it stores two entries of 16 bytes each.
5. Directory entries contain inode numbers and filenames, while file metadata is stored separately in inodes.

4 Part 2 – Implementing mystat and Hard Links

4.1 Creating the mystat Program

The second part required implementing a user program named `mystat`.

The program uses the `stat()` system call to obtain information about files.

The source file was created using:

```
nano user/mystat.c
```

The implementation was:

```
#include "kernel/types.h"
#include "kernel/stat.h"
#include "user/user.h"

char *
typename(int t)
{
    if(t == T_DIR)
        return "directory";

    if(t == T_FILE)
        return "file";

    if(t == T_DEVICE)
        return "device";

    return "?";
}

int
main(int argc, char *argv[])
{
    struct stat st;

    for(int i = 1; i < argc; i++){

        if(stat(argv[i], &st) < 0){
            fprintf(2,
                "mystat: cannot stat %s\n",
                argv[i]);
            continue;
        }

        printf("%s: %s, inode %d, "
               "links %d, size %ld bytes\n",
               argv[i],
               typename(st.type),
               st.ino,
               st.nlink,
               st.size);
```

```
    }

    exit(0);
}
```

The function `typename()` converts numeric file types into readable names.
The `stat()` call fills a `struct stat` with file metadata.
The program prints:

- filename;
- file type;
- inode number;
- number of hard links;
- file size.

4.2 Adding mystat to the Makefile

The program was registered in the `UPROGS` section of the Makefile:

```
$U/_mystat\
```

After saving the Makefile, xv6 was compiled and executed:

```
make qemu
```

The compilation completed successfully.

4.3 Testing mystat

The following command was executed inside xv6:

```
mystat README . console
```

The output was:

```
README: file, inode 2, links 1,
size 2441 bytes

.: directory, inode 1, links 1,
size 1024 bytes

console: device, inode 26, links 1,
size 0 bytes
```

This confirmed that the program correctly identified regular files, directories, and devices.

The inode number of the console device was 26 in this build because the addition of the new program changed the file system image.

```
bianca@ubuntu-lab:~/xv6-riscv$ nano user/mystat.c
bianca@ubuntu-lab:~/xv6-riscv$ make qemu
riscv64-linux-gnu-gcc -Wall -Werror -Wno-unknown-attributes -O -fno-omit-frame-pointer -g
gdb -gdwarf-2 -ffile-prefix-map=/home/bianca/xv6-riscv=. -march=rv64gc -std=gnu99 -MD -mc
model=medany -ffreestanding -fno-common -nostdlib -fno-builtin-strncpy -fno-builtin-strnc
mp -fno-builtin-strlen -fno-builtin-memset -fno-builtin-memmove -fno-builtin-memcmp -fno-
builtin-log -fno-builtin-bzero -fno-builtin-strchr -fno-builtin-exit -fno-builtin-malloc
-fno-builtin-putc -fno-builtin-free -fno-builtin-memcpy -Wno-main -fno-builtin-printf -fn
o-builtin-fprintf -fno-builtin-vprintf -I. -fno-stack-protector -fno-pie -no-pie   -c -o
user/mystat.o user/mystat.c
riscv64-linux-gnu-ld -z max-page-size=4096 -T user/user.ld -o user/_mystat user/mystat.o
user/ulib.o user/usys.o user/printf.o user/umalloc.o
riscv64-linux-gnu-objdump -S user/_mystat > user/mystat.asm
riscv64-linux-gnu-objdump -t user/_mystat | sed '1,/SYMBOL TABLE/d; s/ .* / /; /^$/d' > u
ser/mystat.sym
mkfs/mkfs fs.img README user/_cat user/_echo user/_forktest user/_grep user/_init user/_k
ill user/_ln user/_ls user/_mkdir user/_rm user/_sh user/_stressfs user/_usertests user/_
grind user/_wc user/_zombie user/_logstress user/_forphan user/_dorphan user/_sync user/_
sleep user/_args user/_mystat
nmeta 47 (boot, super, log blocks 31, inode blocks 13, bitmap blocks 1) blocks 1953 total
 2000
balloc: first 1092 blocks have been allocated
balloc: write bitmap block at sector 46
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m 128M -smp 3 -nograp
hic -global virtio-mmio.force-legacy=false -drive file=fs.img,if=none,format=raw,id=x0 -d
evice virtio-blk-device,drive=x0,bus=virtio-mmio-bus.0

xv6 kernel is booting

hart 1 starting
hart 2 starting
init: starting sh
$ mystat README . console
README: file, inode 2, links 1, size 2441 bytes
.: directory, inode 1, links 1, size 1024 bytes
console: device, inode 26, links 1, size 0 bytes
$
```

Figure 2: Compilation and execution of the mystat program

4.4 Creating a Hard Link

The next experiment investigated how hard links work in xv6.

A new hard link was created:

```
ln README r2
mystat README r2
```

The output was:

```
README: file, inode 2, links 2,
size 2441 bytes

r2: file, inode 2, links 2,
size 2441 bytes
```

Both filenames referenced inode number 2.

The link count increased from 1 to 2 because two directory entries now pointed to the same inode.

The file size remained unchanged.

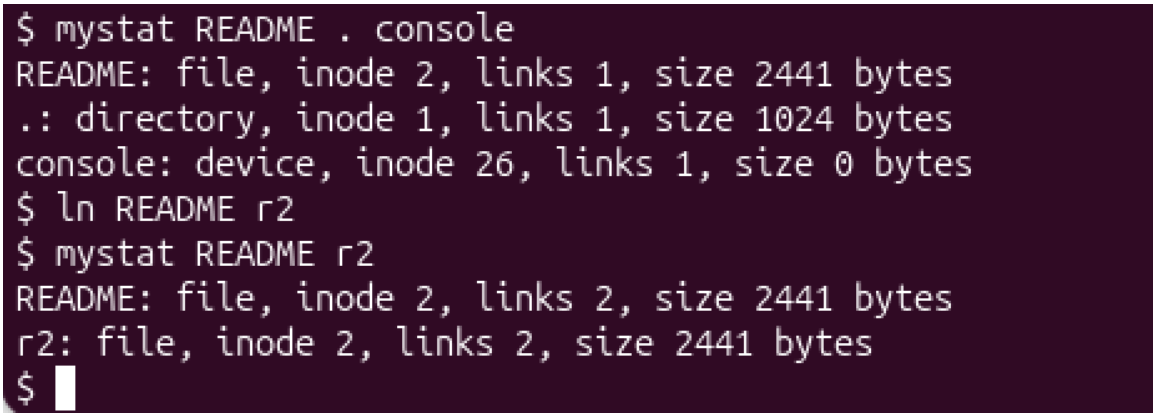


Figure 3: Creating a hard link and inspecting inode metadata

4.5 Removing the Original Filename

The original filename was removed using:

```
rm README
mystat r2
cat r2 | wc
```

The resulting output was:

```
r2: file, inode 2, links 1,
size 2441 bytes

48 336 2441
```

The number of hard links decreased from 2 to 1.

However, the inode number and file size remained unchanged.

The command `cat r2 | wc` confirmed that the contents were still accessible.

This demonstrates that removing a filename does not immediately delete the underlying file data while another hard link exists.

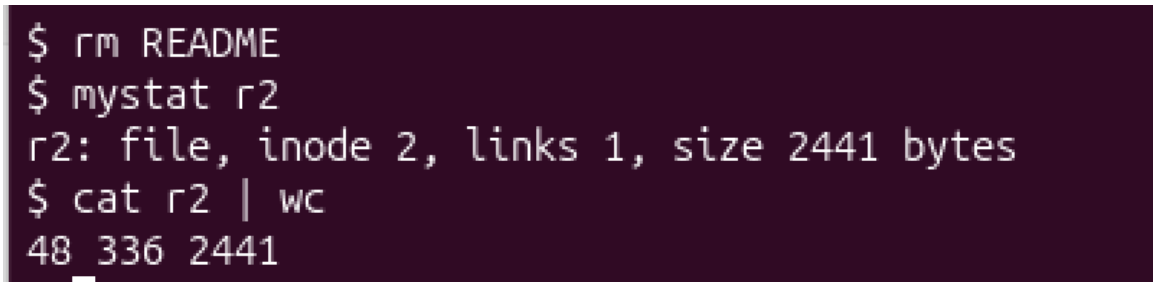


Figure 4: Removing README and accessing the file through r2

4.6 Examining Link Count Changes

The system call responsible for removing directory entries is implemented in `kernel/sysfile.c`.

The following command was used:

```
grep -n "nlink--" kernel/sysfile.c
```

The output was:

```
178: ip->nlink--;
239: dp->nlink--;
244: ip->nlink--;
```

Inside `sys_unlink()`, the statement:

```
ip->nlink--;
```

decreases the inode link count.

Data blocks are freed when the inode has no remaining directory links and no active references.

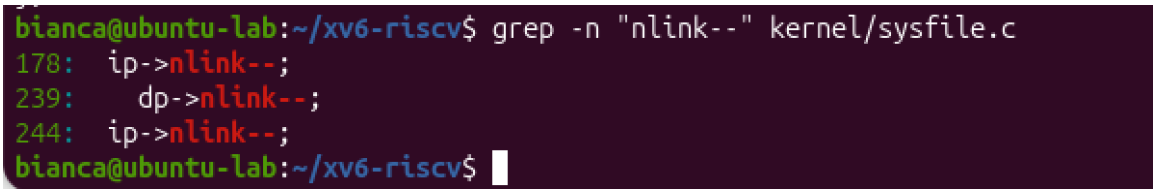


Figure 5: Locating link-count updates in kernel/sysfile.c

4.7 Examining the On-Disk Inode Structure

The inode format was examined in `kernel/fs.h`.

The structure contains:

```
struct dinode {
    short type;
    short major;
    short minor;
    short nlink;
    uint size;
    uint addrs[NDIRECT + 1];
};
```

The fields have the following meanings:

Field	Purpose
type	Identifies the file type
major	Major device number
minor	Minor device number
nlink	Number of hard links
size	File size in bytes
addrs	Addresses of data blocks

The inode stores metadata and block addresses rather than the ordinary filename.

4.8 Part 2 – Observations

1. The `mystat` program successfully displayed file metadata.
2. Hard links share the same inode.

3. Creating a hard link increased the link count from 1 to 2.
4. Removing the original filename reduced the link count to 1.
5. The file contents remained accessible through the remaining hard link.
6. The inode structure stores file type, device numbers, link count, size, and block addresses.

5 Part 3 – Measuring File System Limits

5.1 Examining File System Constants

The limits of the xv6 file system were investigated by examining `kernel/fs.h`.

The command used was:

```
grep -n "define BSIZE\|define NDIRECT\
\|define NINDIRECT\|define DIRSIZ" kernel/fs.h
```

The constants were:

```
#define BSIZE 1024
#define NDIRECT 12
#define NINDIRECT (BSIZE / sizeof(uint))
#define DIRSIZ 14
```

Their values and meanings are:

Constant	Value	Meaning
BSIZE	1024	Disk block size in bytes
NDIRECT	12	Direct data block pointers
NINDIRECT	256	Indirect block entries
DIRSIZ	14	Directory name length in bytes

5.2 Calculating Maximum File Size

An xv6 inode has 12 direct block pointers and one single-indirect block pointer.

Each disk block contains 1024 bytes.

A block address occupies 4 bytes.

Therefore, the indirect block can contain:

```
NINDIRECT = 1024 / 4
          = 256
```

The direct blocks can store:

```
12 * 1024 = 12288 bytes
```

The indirect blocks can store:

```
256 * 1024 = 262144 bytes
```

The maximum file size is:

```
MAXFILE = (12 + 256) * 1024

MAXFILE = 274432 bytes
        = 268 KiB
```

Thus, the theoretical maximum file size supported by this inode layout is **274432 bytes**.

5.3 Examining the Disk Layout

The file system image was rebuilt using the following commands:

```
make clean
make fs.img 2>&1 | grep nmeta
```

The output was:

```
nmeta 47 (boot, super, log blocks 31,
inode blocks 13, bitmap blocks 1)
blocks 1953 total 2000
```

The disk layout was:

Component	Blocks
Total disk blocks	2000
Metadata blocks	47
Log blocks	31
Inode blocks	13
Bitmap blocks	1
Data blocks	1953

The metadata also includes one boot block and one superblock.
The number of metadata blocks was:

```
1 + 1 + 31 + 13 + 1 = 47
```

The remaining blocks were:

```
2000 - 47 = 1953
```

These blocks were available for data storage.

```
bianca@ubuntu-lab:~/xv6-riscv$ make clean
rm -f *.tex *.dvi *.idx *.aux *.log *.ind *.ilg \
*/*.o */*.d */*.asm */*.sym \
kernel/kernel fs.img \
mkfs/mkfs .gdbinit \
        user/usys.S \
user/_cat user/_echo user/_forktest user/_grep user/_init user/_kill user/_ln user/_ls us
er/_mkdir user/_rm user/_sh user/_stressfs user/_usertests user/_grind user/_wc user/_zom
bie user/_logstress user/_forphan user/_dorphan user/_sync user/_sleep user/_args user/_m
ystat
bianca@ubuntu-lab:~/xv6-riscv$ make fs.img 2>&1 | grep nmeta
nmeta 47 (boot, super, log blocks 31, inode blocks 13, bitmap blocks 1) blocks 1953 total
 2000
bianca@ubuntu-lab:~/xv6-riscv$
```

Figure 6: Rebuilding the xv6 file system image and inspecting disk metadata

5.4 Testing Long Filenames

The xv6 file system stores only 14 bytes for a directory entry name.

To test this limitation, the following command was executed:

```
mkdir averyveryverylongname
ls
```

The resulting directory appeared as:

```
averyveryveryl 1 27 32
```

The original name was longer than 14 characters.

Because `DIRSIZ` is 14, the displayed name was truncated to:

```
averyveryveryl
```

This confirmed the filename storage limit defined in `kernel/fs.h`.

```
$ mkdir averyveryverylongname
$ ls
.              1 1 1024
..             1 1 1024
README         2 2 2441
cat            2 3 36728
echo           2 4 35592
forktest       2 5 18136
grep           2 6 44184
init           2 7 35976
kill           2 8 35512
ln             2 9 35304
ls             2 10 42880
mkdir          2 11 35568
rm             2 12 35552
sh             2 13 58312
stressfs       2 14 36416
usertests      2 15 206800
grind          2 16 51824
wc             2 17 37728
zombie         2 18 34848
logstress      2 19 37560
forphan        2 20 36352
dorphan        2 21 35800
sync           2 22 34944
sleep          2 23 35280
args           2 24 35272
mystat         2 25 36320
console        3 26 0
averyveryveryl 1 27 32
$
```

Figure 7: Testing the maximum filename length in xv6

5.5 Part 3 – Observations

1. The disk block size was 1024 bytes.
2. Each inode contained 12 direct block addresses and one indirect pointer.
3. The indirect block stored 256 block addresses.
4. The theoretical maximum file size was 274432 bytes.
5. The file system image contained 2000 blocks, of which 1953 were available as data blocks.
6. Directory entry names were limited to 14 bytes, causing longer names to be truncated.

6 Part 4 – Implementing Recursive Find

6.1 Creating the find Program

The final programming task was to implement a recursive file search utility.

The program accepts two arguments:

```
find <directory> <name>
```

It recursively searches the specified directory and prints paths whose final component matches the requested name.

The source file was created using:

```
nano user/find.c
```

The implementation was:

```
#include "kernel/types.h"
#include "kernel/stat.h"
#include "kernel/fs.h"
#include "user/user.h"

void
find(char *path, char *name)
{
    char buf[512];
    char *p;
    int fd;
    struct dirent de;
    struct stat st;

    if((fd = open(path, 0)) < 0){
        fprintf(2,
            "find: cannot open %s\n",
            path);
        return;
    }
```

```
if(fstat(fd, &st) < 0){
    fprintf(2,
        "find: cannot stat %s\n",
        path);
    close(fd);
    return;
}

if(st.type != T_DIR){
    close(fd);
    return;
}

if(strlen(path) + 1 + DIRSIZ + 1
   > sizeof(buf)){
    fprintf(2, "find: path too long\n");
    close(fd);
    return;
}

strcpy(buf, path);
p = buf + strlen(buf);
*p++ = '/';

while(read(fd, &de, sizeof(de))
      == sizeof(de)){

    if(de.inum == 0)
        continue;

    if(strcmp(de.name, ".") == 0 ||
       strcmp(de.name, "..") == 0)
        continue;

    memmove(p, de.name, DIRSIZ);
    p[DIRSIZ] = 0;

    if(stat(buf, &st) < 0){
        fprintf(2,
            "find: cannot stat %s\n",
            buf);
        continue;
    }

    if(strcmp(de.name, name) == 0)
        printf("%s\n", buf);

    if(st.type == T_DIR)
        find(buf, name);
}
```

```
    close(fd);
}

int
main(int argc, char *argv[])
{
    if(argc != 3){
        fprintf(2,
            "Usage: find <dir> <name>\n");
        exit(1);
    }

    find(argv[1], argv[2]);

    exit(0);
}
```

6.2 Explanation of the Program

The program uses several file system functions provided by xv6.

Function	Purpose
open()	Opens the directory
fstat()	Reads metadata from the opened file
read()	Reads directory entries
strcmp()	Compares entry names
stat()	Retrieves metadata for an entry
close()	Closes an open file descriptor

The program checks whether each entry is a file or a directory.

When a matching filename is found, its complete path is printed.

If the entry is a directory, the `find()` function calls itself to continue the search recursively.

The program ignores `.` and `..` to prevent infinite recursion.

Entries with inode number zero are skipped because they represent unused directory slots.

Every opened directory is eventually closed to avoid exhausting available file descriptors.

6.3 Adding find to the Makefile

The following line was added to the `UPROGS` section:

```
$U/_find\
```

The two custom programs were registered together:

```
$U/_mystat\
$U/_find\
```

The project was rebuilt using:

```
make clean
make qemu
```

The program compiled successfully and was included in the xv6 file system image.

6.4 Creating the Test Directory Tree

The following commands were executed:

```
mkdir a
mkdir a/b

echo x > a/b/r2
echo y > a/r2
ln cat r2
```

The resulting structure was:

```
.
|-- a
| |-- b
| | `-- r2
| `-- r2
`-- r2
```

The root-level file `r2` was created as a hard link to `cat`.

The other two files were created using the `echo` command.

6.5 Testing Recursive File Search

The following command was executed:

```
find . r2
```

The output was:

```
./a/b/r2
./a/r2
./r2
```

All three matching paths were found.

This confirmed that the program could search the root directory and nested subdirectories successfully.

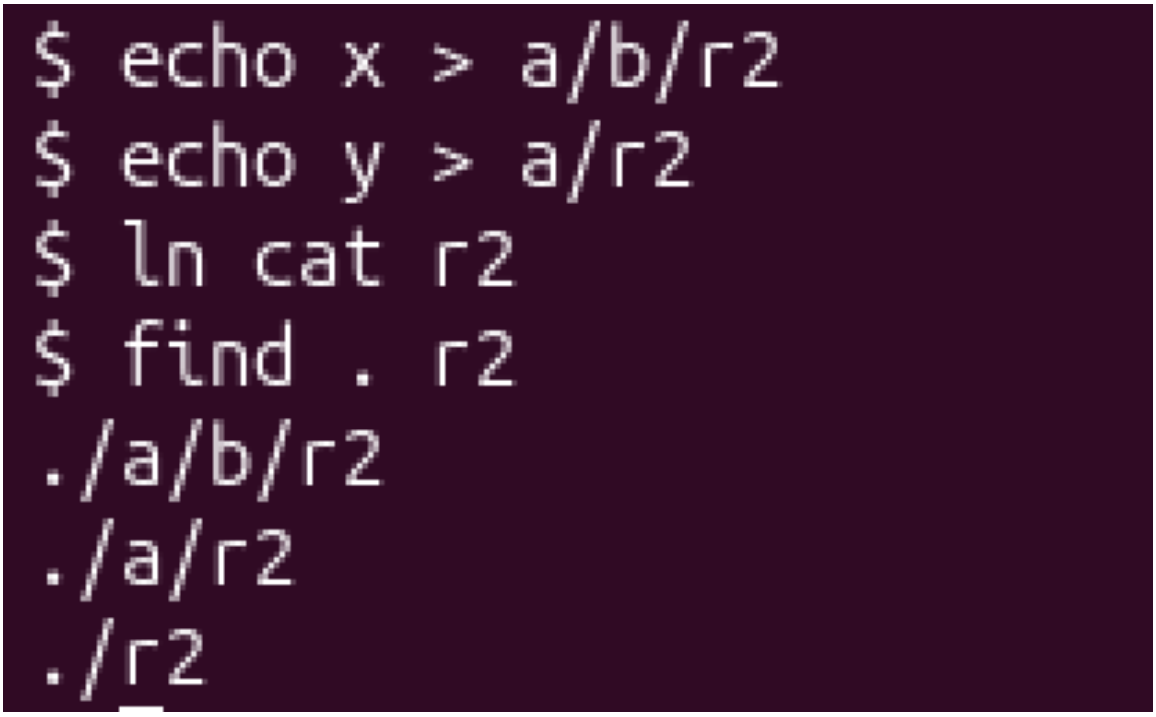


Figure 8: Recursive search for files named r2

6.6 Testing Directory Search

The program was also tested by searching for a directory named `b`.

The command was:

```
find a b
```

The output was:

```
a/b
```

The program successfully identified the matching directory.

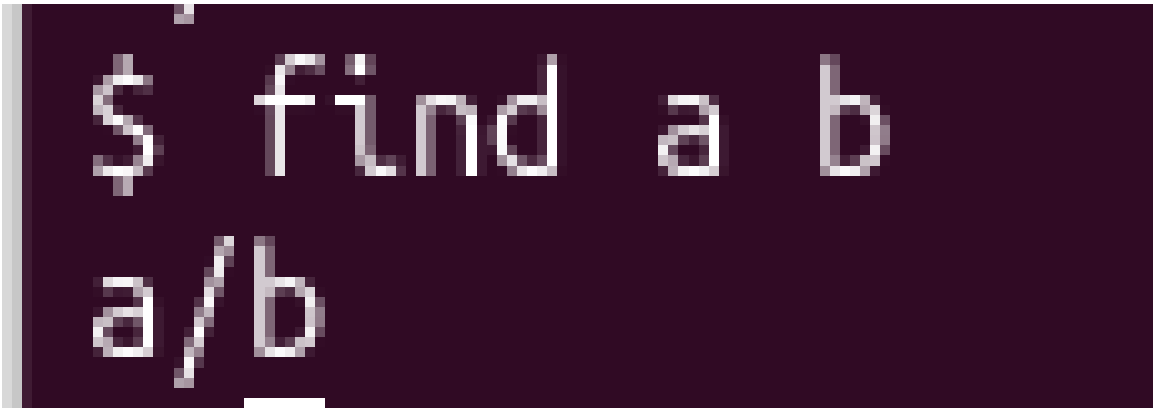


Figure 9: Recursive find program locating directory b

6.7 Part 4 – Observations

1. The `find` program compiled successfully and ran inside xv6.

2. The recursive search located all three files named `r2`.
3. The program also successfully located a directory named `b`.
4. The entries `.` and `..` were skipped to avoid recursive loops.
5. Closing file descriptors was important because xv6 limits the number of open files per process.
6. Directory traversal was implemented using `open()`, `read()`, `stat()`, and recursion.

7 Results and Discussion

The laboratory demonstrated the main internal operations of the xv6 file system.

The `mystat` program successfully retrieved file metadata and displayed file types, inode numbers, link counts, and sizes.

The hard-link experiment confirmed that multiple filenames can reference the same inode.

Removing one filename decreased the link count but did not remove the file data while another hard link remained.

The file system limits were determined by examining the constants in `kernel/fs.h`.

The maximum filename length was 14 bytes, while the maximum file size supported by the inode block-addressing scheme was 274432 bytes.

Finally, the `find` program demonstrated recursive directory traversal.

Both file and directory searches produced the expected results.

8 GitHub Repository

The laboratory files were organized in the directory:

```
/home/bianca/os-lab6
```

The following files were prepared:

```
Makefile
mystat.c
find.c
notes.md
```

The local Git repository was initialized using:

```
cd ~/os-lab6
git init
git branch -M main
git add .
git commit -m "Operating Systems Lab 6"
```

The remote repository was configured using:

```
git remote add origin \
https://github.com/Biancatataroi/\
Operating-Systems-Lab6.git
```

The files were uploaded using:

```
git push -u origin main
```

Git confirmed the successful upload:

```
[new branch] main -> main

branch 'main' set up to track
'origin/main'.
```

The repository contains the laboratory source code, Makefile, and notes.
The GitHub repository is available at:

https://github.com/Biancatataroi/Operating-Systems-Lab6

9 Conclusion

In this laboratory work, I investigated the internal structure and behavior of the xv6 file system through practical experiments and C programming.

The first part focused on understanding how xv6 represents files and directories. By using the `ls` command, I observed file types, inode numbers, and file sizes. I also created a new directory and examined the special entries `.` and `..`.

The second part involved implementing the `mystat` program, which uses the `stat()` system call to retrieve information about files. The program successfully displayed file types, inode numbers, hard-link counts, and file sizes.

The experiments with hard links demonstrated how multiple filenames can reference the same inode. After removing the original filename, the remaining hard link continued to provide access to the file data. This helped me understand the relationship between directory entries, inodes, and link counts.

The third part focused on file system limitations. I examined the constants defined in `kernel/fs.h` and calculated the maximum file size using direct and indirect block addressing.

I also investigated the disk layout and observed how the file system divides available blocks between metadata and file data.

The experiment with long filenames confirmed that xv6 stores only 14 bytes for each directory entry name.

Finally, the fourth part involved implementing a recursive `find` program. The program searched directories, processed directory entries, and located files and subdirectories with matching names.

The successful tests demonstrated how recursion can be used together with file system calls to traverse a directory hierarchy.

Overall, this laboratory work provided a practical understanding of inodes, hard links, directories, file metadata, disk allocation, and recursive file system operations.

It also improved my ability to read operating system source code, implement user programs in C, and test them inside xv6.